CASE STUDY 11

Examination Result and Grace-Mark

1. PROBLEM STATEMENT

Process marks in multiple subjects, apply subject-wise pass marks and a grace-mark policy, and generate a final result
without exceeding maximum marks.

2. OBJECTIVES

• Check subject-wise pass conditions.

• Apply grace marks only within the allowed policy.

• Calculate total, percentage, and result.

3. ASSUMPTIONS

• There are three subjects, each out of 100.

• A student must score at least 35 in every subject.

• If exactly one subject is below 35 by no more than 5 marks, grace marks may raise it to 35.

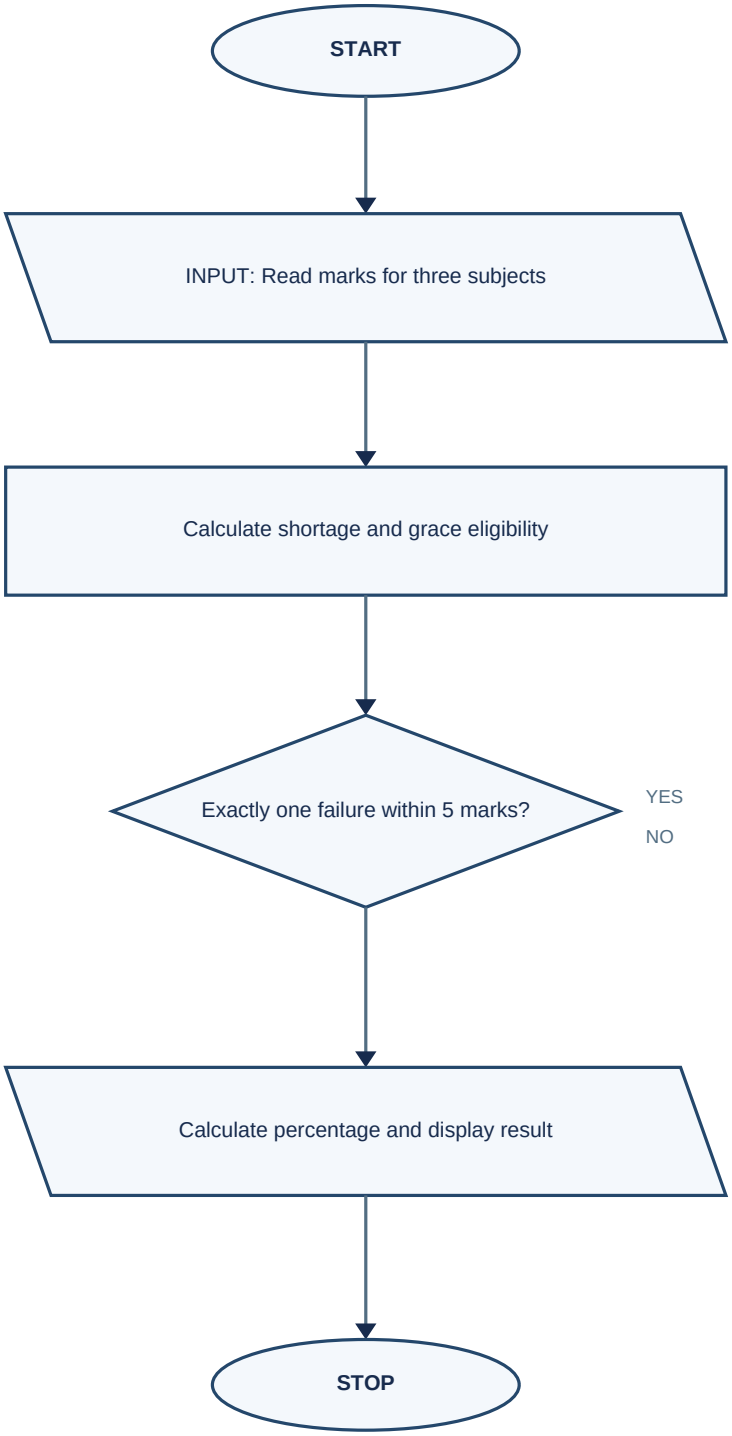
• Grace marks cannot increase a subject above 100.

4. ALGORITHM

1. Read marks for the three subjects and validate each mark is between 0 and 100.
2. Find the subjects whose marks are below 35.
3. If exactly one subject has failed, calculate its shortage and apply grace only when shortage is at most 5.
4. Calculate total marks and percentage after any permitted grace.
5. Check all subjects again and display PASS or FAIL with the result details.

5. FLOWCHART

Oval: START/STOP | Parallelogram: input/output | Rectangle: process/calculation | Diamond: decision



6. PSEUDOCODE

```
BEGIN
INPUT marks for three subjects
VALIDATE every mark is between 0 and 100
FIND subjects below pass_mark 35
IF exactly one subject failed
CALCULATE shortage
IF shortage <= 5: add required grace marks
CALCULATE total and percentage
IF every subject >= 35: result = PASS
ELSE: result = FAIL
DISPLAY marks, grace used, total, percentage and result
END
```

7. PYTHON SOURCE CODE

```
marks = [32, 70, 65]
pass_mark = 35
grace_limit = 5
failed = []
for i in range(len(marks)):
if marks[i] < pass_mark:
failed.append(i)
grace_used = 0
if len(failed) == 1:
index = failed[0]
shortage = pass_mark - marks[index]
if shortage <= grace_limit:
grace_used = shortage
marks[index] = marks[index] + shortage
total = sum(marks)
percentage = total / len(marks)
if all(mark >= pass_mark for mark in marks):
result = "PASS"
else:
result = "FAIL"
print("FINAL MARKS:", marks)
print("Grace marks used:", grace_used)
print("Total:", total, "/ 300")
print("Percentage:", round(percentage, 2), "%")
print("Result:", result)
```

8. TEST CASES & EXPECTED RESULTS

TEST INPUT / CONDITION REASONING EXPECTED RESULT

32, 70, 65 One subject is short by 3; grace allowed [35, 70, 65], PASS

25, 70, 65 Short by 10; grace not allowed FAIL

30, 32, 80 Two failed subjects FAIL

9. SAMPLE OUTPUT

```
FINAL MARKS: [35, 70, 65]
Grace marks used: 3
Total: 170 / 300
Percentage: 56.67 %
Result: PASS
```

10. CONCLUSION

The program validates marks, applies the limited grace policy, and produces a final result while respecting the maximum-mark limit.

SYLLABUS CONCEPTS Variables • Data Types • Operators • Conditions • Loops • Lists • Nested Lists • Tuples • String Operations